

M1 DATA ENGINEERING &amp; IA — EFREI PARIS

# Crypto Market Monitor

Surveillance temps réel des marchés crypto via pipeline Kafka, analytics en fenêtre glissante et dashboard live Socket.IO.

---

|           |                                                |
|-----------|------------------------------------------------|
| AUTEURS   | Adam Beloucif & Emilien Morice                 |
| FORMATION | M1 Data Engineering & IA — EFREI Paris         |
| MODULE    | Real-Time Engineering — S9 2025-2026           |
| CONTEXTE  | Réalisé à 2 personnes (consigne prévue pour 4) |
| DATE      | Juin 2026                                      |

Apache Kafka 3.7

Node.js + TypeScript

Socket.IO

Docker

Chart.js

# Contexte & objectifs

Ce rapport présente l'architecture, les choix techniques et les résultats du projet Crypto Market Monitor, réalisé dans le cadre du module Real-Time Engineering du M1 Data Engineering & IA de l'EFREI Paris (2025-2026). Le projet a été réalisé en binôme alors que la consigne prévoyait une équipe de quatre personnes.

L'objectif : construire un pipeline de streaming bout-en-bout qui ingère des transactions crypto en direct depuis Binance et Coinbase, les traite via Apache Kafka, calcule des métriques analytiques en fenêtre glissante et pousse les résultats vers un dashboard live via Socket.IO.

**100M+**

transactions/jour  
sur Binance en 2024

**< 1s**

latence cible  
pour la détection  
d'anomalie

**3**

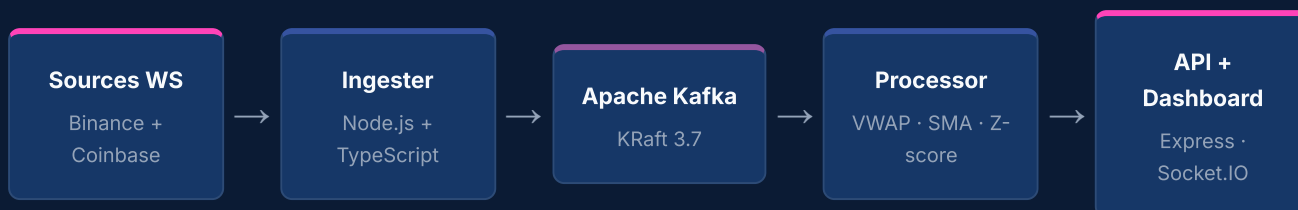
sources WebSocket  
Binance + Coinbase

**4**

services  
indépendants  
orchestrés via  
Docker

# Pipeline **Kafka** bout-en-bout

Le système suit un pipeline linéaire : collecte WebSocket → Kafka → traitement analytique → API REST/WebSocket → dashboard. Chaque service est isolé dans un conteneur Docker indépendant avec healthcheck et restart policy.



## Composants

| SERVICE   | RÔLE                                                            | TECHNOLOGIES                     |
|-----------|-----------------------------------------------------------------|----------------------------------|
| Ingestor  | Clients WS Binance + Coinbase, normalisation, production Kafka  | Node.js, TypeScript, ws, kafkajs |
| Kafka     | Tampon de découplage, tolérance aux pannes, multi-consommateurs | Apache Kafka 3.7 KRaft           |
| Processor | Consomme les trades, calcule VWAP, SMA-20, z-score, variance    | Node.js, TypeScript, kafkajs     |
| API       | Agrégation, endpoints REST, push WebSocket live                 | Express, Socket.IO, helmet       |
| Dashboard | Graphiques Chart.js, flux anomalies, i18n FR/EN                 | HTML, CSS, JS, Chart.js 4        |

# Mode **KRaft** sans Zookeeper

Kafka 3.7 en mode KRaft élimine la dépendance à Zookeeper. Les métadonnées du cluster sont gérées via le protocole Raft interne, ce qui simplifie le déploiement à **1 conteneur + 1 volume** et réduit la surface d'attaque opérationnelle.

## POURQUOI KAFKA ?

- **Découpling**  
producteurs /  
consommateurs  
: débits  
indépendants
- **Tolérance aux pannes** :  
relecture depuis  
le dernier offset
- **Multi-consommateurs**  
: plusieurs  
groupes  
indépendants
- **Partitionnement**  
par symbole :  
ordre des  
messages  
garanti
- **Retention**  
configurable : 1h  
par défaut dans  
ce projet

## CONFIGURATION KRAFT

```
KAFKA_CFG_PROCESS_ROLES=broker,controller  
KAFKA_CFG_NODE_ID=1  
KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=1@kafka:9093  
KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093
```

Image : `bitnami/kafka:3.7`

## Topics

| TOPIC                 | DESCRIPTION                                       | PARTITIONS | RÉTENTION |
|-----------------------|---------------------------------------------------|------------|-----------|
| <b>crypto-trades</b>  | Transactions brutes normalisées (interface Trade) | 3          | 1h        |
| <b>crypto-metrics</b> | Métriques calculées par symbole (VWAP, SMA...)    | 3          | 1h        |

# Services TypeScript

## Ingesteur — Collecte WebSocket

L'ingester maintient des connexions WebSocket persistantes et normalise chaque message vers une interface Trade commune avant de le produire sur Kafka avec le symbole comme clé de partition.

- **Binance** : stream combiné BTC/USDT + ETH/USDT
- **Coinbase** : BTC-USD via Advanced Trade WebSocket
- **Reconnexion auto** : backoff exponentiel, max 10 tentatives, délai max 30s
- **Arrêt propre** SIGTERM/SIGINT avec vidage du buffer Kafka

## Processor — Analytique temps réel

Buffer circulaire en mémoire (max 1000 trades / symbole, éviction TTL 10 min). Recalcul à chaque nouveau trade.

| MÉTRIQUE   | FORMULE                                                              | FENÊTRE   | OBJECTIF                      |
|------------|----------------------------------------------------------------------|-----------|-------------------------------|
| VWAP       | $\text{sum}(\text{prix} \times \text{qte}) / \text{sum}(\text{qte})$ | 1 min     | Prix moyen pondéré par volume |
| SMA-20     | moyenne 20 derniers prix                                             | 20 trades | Tendance lissée               |
| Z-Score    | $(\text{val} - \mu) / \sigma$                                        | 1 min     | Anomalie si $> 2.5\sigma$     |
| Var. 1min  | $(\text{prix} - \text{prix}_0) / \text{prix}_0$                      | 1 min     | Performance instantanée       |
| Var. 5min  | $(\text{prix} - \text{prix}_0) / \text{prix}_0$                      | 5 min     | Performance moyen terme       |
| High / Low | max / min des prix                                                   | 1 min     | Amplitude de la bougie        |

## API Server — REST + Socket.IO

## ENDPOINTS REST

```
GET /api/health GET /api/metrics
GET /api/metrics/:symbol GET
/api/history/:symbol GET
/api/anomalies
```

## ÉVÉNEMENTS SOCKET.IO

- **metrics:update** — nouvelles métriques
- **anomaly:detected** — alerte z-score
- **server:stats** — connexions / msg/s
- **initial:state** — état complet à la connexion
- **subscribe:symbol** — choix de la paire

# Interface temps réel

Application vanilla HTML/CSS/JS connectée via Socket.IO. Aucun framework front-end pour minimiser les dépendances et maximiser la vitesse de chargement.

## COMPOSANTS VISUELS

- **Prix hero** avec flash vert/rouge au changement
- **Badges** variation 1min et 5min
- **4 stat-cards** : Volume, Trades, High, Low
- **Chart.js** : ligne prix + SMA-20 superposée
- **Barres** de volume par paire
- **Flux anomalies** + beep AudioContext

## DESIGN & UX

- **Palette EFREI** : navy #163767 + rose #ff43b8
- **Logos SVG** officiels Bitcoin, Ethereum, Coinbase
- **Barre accent** EFREI gradient en haut de page
- **i18n FR/EN** avec bascule localStorage
- **Mode démo** si backend inaccessible (3s)
- **Inter + JetBrains Mono**, reduced-motion

# Production-ready

| COMPOSANT          | PROTECTION                                                                            |
|--------------------|---------------------------------------------------------------------------------------|
| helmet             | HTTP headers : CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy    |
| express-rate-limit | 100 req / 15min / IP — protection brute-force et DDoS applicatif                      |
| zod                | Validation stricte des variables d'environnement — arrêt si manquante ou invalide     |
| CORS strict        | Whitelist explicite des origines autorisées — pas de wildcard (*) en production       |
| morgan             | Logging HTTP avec rotation — aucune stack trace exposée côté client                   |
| Docker non-root    | Tous les conteneurs tournent en utilisateur appuser (UID 1000)                        |
| .dockerignore      | Exclusion des .env, node_modules et secrets des images Docker                         |
| Secrets env vars   | Zéro secret en dur dans le code — .env gitignored, patterns défensifs dans .gitignore |



# 3 options Docker

## 01

### Lanceur autonome

```
node
launcher/src/index.mjs
Vérifie Docker, build
toutes les images, attend
les healthchecks et ouvre
le dashboard
automatiquement. Compile
en .exe standalone : npm
run build:win
```

## 02

### Image tout-en-un

```
docker build \ -f all-in-
one.Dockerfile \ -t
crypto-monitor . docker
run -p 8080:8080 \
crypto-monitor 1
conteneur supervisord :
Kafka + 3 services +
nginx
```

## 03

### Docker Compose

```
cp .env.example .env
docker-compose up --build
-d Dashboard :8080 Kafka
UI :8090 API :3001 6
services indépendants
avec healthchecks
```

## Variables d'environnement clés

| VARIABLE                 | DÉFAUT         | DESCRIPTION                    |
|--------------------------|----------------|--------------------------------|
| KAFKA_BROKERS            | localhost:9092 | Bootstrap servers Kafka        |
| KAFKA_TOPIC_TRADES       | crypto-trades  | Topic des transactions brutes  |
| KAFKA_TOPIC_METRICS      | crypto-metrics | Topic des métriques calculées  |
| API_PORT                 | 3001           | Port du serveur API            |
| ANOMALY_ZSCORE_THRESHOLD | 2.5            | Seuil z-score pour les alertes |
| HISTORY_WINDOW_SECONDS   | 300            | Rétention historique (5 min)   |

# Résultats & perspectives

## LIVRÉ

- Pipeline Kafka bout-en-bout fonctionnel
- 3 services TypeScript production-ready
- Dashboard EFREI-branded FR/EN + mode démo
- Sécurité : helmet, rate-limit, zod, non-root
- 3 options déploiement dont .exe standalone
- Réalisé à 2 pour une consigne de 4

## ÉVOLUTIONS POSSIBLES

- Nouvelles paires : SOL, BNB, XRP...
- Alertes email/SMS via Kafka Connect
- InfluxDB / TimescaleDB pour historique
- Kubernetes + Helm pour la scalabilité
- Auth JWT REST + Socket.IO
- Tests d'intégration Testcontainers